

PROYECTO 1 · INFORME

Torneo de Estrategias del Dilema del Prisionero Iterado

Simulación en C++ con programación estructurada

Integrantes del grupo

#	Nombre completo	Código	% de participación
1			
2			
3			
4			
5			

Fecha de entrega: _____

Contenido

1. Introducción
 2. Descripción de las estrategias
 3. Diseño del programa
 4. Análisis de resultados
 5. Conclusiones
 6. Bibliografía y fuentes
- Anexo: compilación y pruebas

1. Introducción

El **Dilema del Prisionero** es un problema clásico de la teoría de juegos. Dos participantes deciden, sin comunicarse, si **cooperan (C)** o **traicionan (T)**. En una sola ronda traicionar siempre da más puntos que cooperar: 5 contra 3 si el otro coopera y 1 contra 0 si el otro traiciona. Sin embargo, si ambos traicionan obtienen 1 punto cada uno, mucho menos que los 3 que habrían obtenido cooperando.

Jugador A \ Jugador B	B coopera (C)	B traiciona (T)
A coopera (C)	A = 3, B = 3	A = 0, B = 5
A traiciona (T)	A = 5, B = 0	A = 1, B = 1

En el **Dilema del Prisionero Iterado** el juego se repite muchas rondas contra el mismo rival. Como cada jugador recuerda lo que hizo el otro, aparecen estrategias que premian la cooperación y castigan la traición, y la mejor decisión deja de ser simplemente traicionar.

El objetivo del proyecto es implementar en C++, con programación estructurada (funciones, estructuras y arreglos, sin clases), un torneo *todos contra todos* en el que 8 estrategias (6 obligatorias y 2 diseñadas por el equipo) se enfrentan en partidas de más de 200 rondas. El programa calcula el puntaje de cada estrategia contra cada rival, sus estadísticas (victorias, derrotas, empates y porcentajes de cooperación y traición) y el ranking final, para determinar la estrategia ganadora y analizar por qué ganó.

2. Descripción de las estrategias

2.1 Estrategias obligatorias

#	Estrategia	Comportamiento	Función en el código
1	Siempre Cooperar	Juega C en todas las rondas.	<code>decidirSiempreCooperar</code>
2	Siempre Traicionar	Juega T en todas las rondas.	<code>decidirSiempreTraicionar</code>
3	Aleatoria	Elige C o T con 50 % de probabilidad (<code>rand() % 2</code>).	<code>decidirAleatoria</code>
4	Tit for Tat	Coopera en la ronda 1; después copia la jugada anterior del oponente.	<code>decidirTitForTat</code>
5	Vengativa	Coopera hasta que el oponente traicione una vez; desde ahí traiciona siempre.	<code>decidirVengativa</code>
6	Ojo por Dos Ojos	Coopera en las rondas 1 y 2; desde la 3 traiciona solo si el oponente traicionó en las dos rondas anteriores.	<code>decidirOjoPorDosOjos</code>

2.2 Estrategia propia 1: Ganar-Mantener, Perder-Cambiar (Pavlov)

Tipo: reactiva. **Descripción:** evalúa el resultado de su última jugada. Si le fue bien (obtuvo 3 o 5 puntos) la repite; si le fue mal (0 o 1 punto) la cambia. En la ronda 1 coopera.

Ronda anterior (yo, oponente)	Puntos obtenidos	Evaluación	Jugada actual
C, C	3	Ganó → mantiene	C
T, C	5	Ganó → mantiene	T
C, T	0	Perdió → cambia	T
T, T	1	Perdió → cambia	C

Algoritmo: solo mira la ronda inmediatamente anterior (`historialPropio[numeroRonda - 2]` y `historialOponente[numeroRonda - 2]`). Las cuatro filas de la tabla se reducen a una sola regla: **si ambos jugaron lo mismo, coopera; si jugaron distinto, traiciona.**

```
if (miJugadaAnterior == jugadaAnteriorRival) return 'C';  
return 'T';
```

Justificación: se diferencia de Tit for Tat porque no copia al oponente, sino que reacciona a su propio resultado. Así puede salir sola de una cadena de traiciones mutuas (después de T-T vuelve a cooperar) y, al mismo tiempo, sigue aprovechando a un rival que no castiga (después de T-C sigue traicionando).

2.3 Estrategia propia 2: Sonda (Prober)

Tipo: mixta. **Descripción:** empieza con una secuencia fija para “probar” al oponente y, según cómo reacciona, decide si coopera con él o si lo explota durante el resto de la partida.

1. Ronda 1: **T**. Ronda 2: **C**. Ronda 3: **C** (fase de exploración).
2. Desde la ronda 4, si el oponente jugó T en la ronda 2 o en la 3 (se defendió de la traición inicial), la Sonda se comporta como Tit for Tat.

3. Si el oponente jugó C en las rondas 2 y 3 (no se defendió), la Sonda traiciona siempre.

```
bool oponenteSeDefendio = (historialOponente[1] == 'T' || historialOponente[2] == 'T');  
if (opponenteSeDefendio) return historialOponente[numeroRonda - 2]; // modo Tit for Tat  
return 'T'; // modo explotar
```

Algoritmo: la variable de estado `opponenteSeDefendio` se calcula a partir del historial. Como las jugadas de las rondas 2 y 3 ya no cambian, el modo queda fijado para el resto de la partida. Se calcula desde el historial (y no con una variable `static`) para que cada partida empiece limpia y no se mezclen rivales distintos.

Justificación: busca sacar el máximo contra estrategias ingenuas, como Siempre Cooperar, sin perder demasiado contra estrategias que castigan, como Tit for Tat. Es diferente de todas las obligatorias porque tiene dos fases: exploración y decisión.

3. Diseño del programa

3.1 Organización general

El programa está dividido en módulos `.h/.cpp`, cada uno con una sola responsabilidad. `main()` solo llama a las funciones en orden y no contiene lógica del juego.

Archivo	Responsabilidad
<code>tipos_torneo.h</code>	Constantes globales, tipo <code>FuncionDecision</code> y <code>struct Estrategia</code> .
<code>lectura_rondas.h/.cpp</code>	Lectura validada del número de rondas por teclado.
<code>estrategias_juego.h/.cpp</code>	Las 8 funciones de decisión e inicialización del arreglo de estrategias.
<code>matriz_pagos.h/.cpp</code>	Puntos de una ronda según la matriz de pagos.
<code>partida_iterada.h/.cpp</code>	Una partida completa entre dos estrategias.
<code>torneo_round_robin.h/.cpp</code>	Torneo todos contra todos (cada par una sola vez).
<code>estadisticas_torneo.h/.cpp</code>	Victorias, derrotas, empates, porcentajes y ranking.
<code>reporte_resultados.h/.cpp</code>	Impresión del detalle, del ranking y de la ganadora.
<code>main_torneo.cpp</code>	Función <code>main()</code> .

3.2 Principales funciones

Función	Qué hace
<code>int leerNumeroRondas()</code>	Pide el número de rondas hasta que sea un entero mayor a 200 (no hay máximo).
<code>void inicializarEstrategias(Estrategia[])</code>	Asigna id, nombre y función a cada estrategia y pone sus contadores en 0.
<code>char decidirXxx(int, const char[], const char[])</code>	Una por estrategia; devuelve 'C' o 'T'.
<code>void calcularPuntosRonda(char, char, int&, int&)</code>	Aplica la matriz de pagos a una ronda.
<code>void ejecutarPartida(Estrategia&, Estrategia&, int, long long&, long long&)</code>	Juega todas las rondas entre dos estrategias, guarda el historial y cuenta las C y T.
<code>void ejecutarTorneo(Estrategia[], int)</code>	Doble bucle <code>i, j = i + 1</code> : 28 partidas sin repetir. Guarda <code>puntosContra</code> y <code>puntajeTotal</code> .
<code>void calcularEstadisticas(Estrategia[])</code>	Calcula victorias, derrotas, empates y porcentajes de C y T.
<code>void ordenarRanking(const Estrategia[], int[])</code>	Ordena los índices por puntaje (burbuja, de mayor a menor).
<code>mostrarDetalleTorneo, mostrarDetalleEstrategia, mostrarRanking, mostrarGanador</code>	Salida por consola.

3.3 Estructuras de datos

- `struct Estrategia`: agrupa todo lo de una estrategia: `id`, `nombre`, `decidir` (puntero a su función de decisión), `puntajeTotal`, `puntosContra[ NUM_ESTRATEGIAS ]`, `vecesC`, `vecesT`, `victorias`, `derrotas`, `empates`, `porcentajeC` y `porcentajeT`. Los puntajes y los contadores de C y T son `long long`: como no hay límite de rondas, con `int` (máximo 2 147 483 647) podrían desbordarse.
- `Estrategia estrategias[ NUM_ESTRATEGIAS ]`: arreglo con los 8 participantes.
- `int ranking[ NUM_ESTRATEGIAS ]`: índices de las estrategias ordenados por puntaje. Se ordenan los índices, no las estructuras, para no perder la correspondencia con `puntosContra`.
- `char *historialA = new char[numeroRondas]`, `char *historialB = new char[numeroRondas]`: jugadas de cada estrategia en una partida. Como el número de rondas no tiene máximo, se crean con memoria dinámica del tamaño exacto al inicio de cada partida y se liberan con `delete[]` al terminarla. La ronda N está en la posición N - 1.
- `typedef char (*FuncionDecision)(int, const char[], const char[])`: permite guardar la función de cada estrategia en su estructura y llamarla desde `ejecutarPartida` sin usar `if / switch` por estrategia.

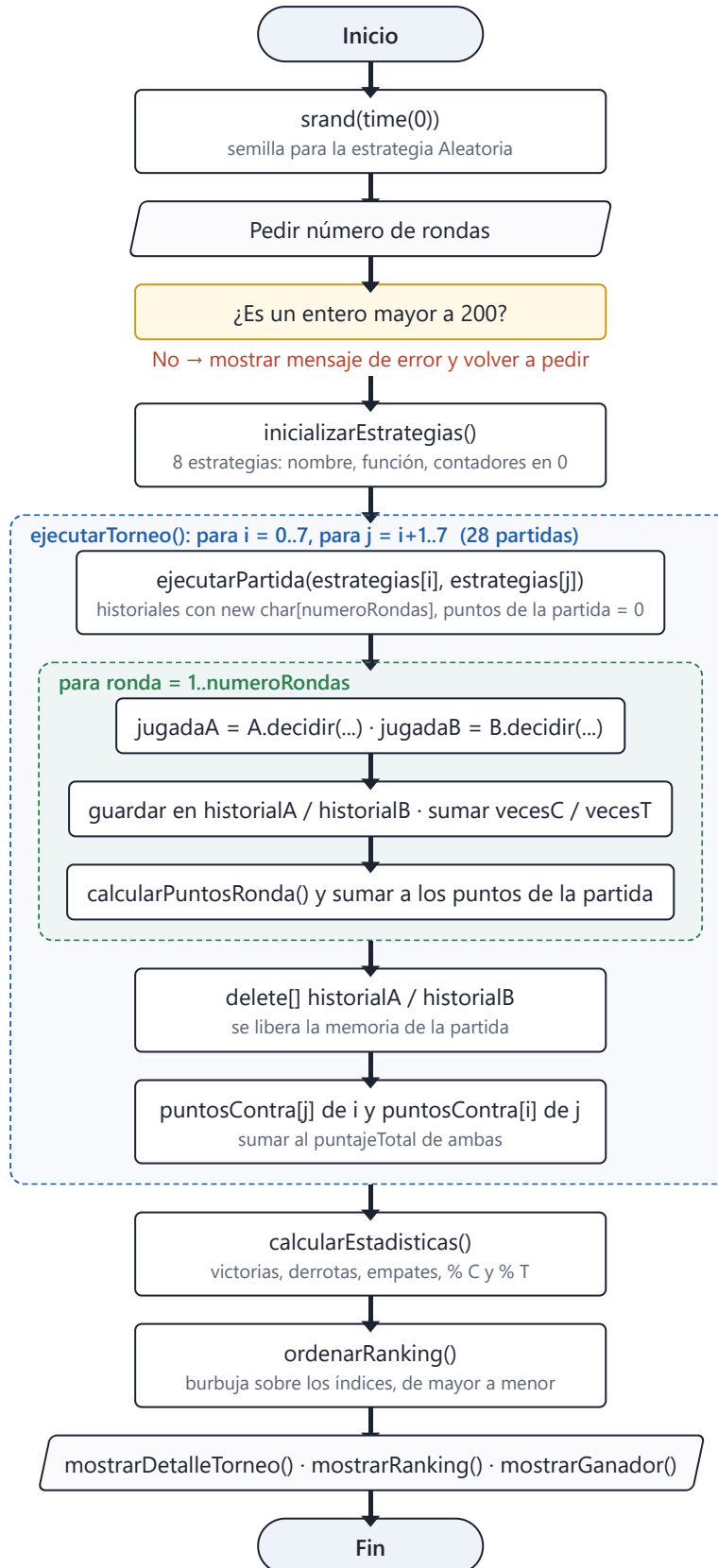
3.4 Variables y constantes importantes

Nombre	Valor	Uso
NUM ESTRATEGIAS	8	Tamaño de los arreglos y límite de los bucles del torneo.
MIN Rondas	201	El enunciado exige más de 200 rondas por partida. No hay máximo.
numeroRondas	lo ingresa el usuario	Rondas de cada partida (mayor a 200, sin límite superior). También es el tamaño de los historiales.
PUNTOS_COOPERACION_MUTUA , PUNTOS_TRAICION_EXITOSA , PUNTOS_ENGANADO , PUNTOS_TRAICION_MUTUA	3, 5, 0, 1	Valores de la matriz de pagos.

3.5 Diagrama de módulos (quién llama a quién)

```
main()
├─ srand(time(0))
├─ leerNumeroRondas()           [lectura_rondas]
├─ inicializarEstrategias()     [estrategias_juego]
├─ ejecutarTorneo()            [torneo_round_robin]
│   └─ ejecutarPartida() x 28 partidas [partida_iterada]
│       ├── estrategia.decidir(...) → decidirXxx() [estrategias_juego]
│       ├── registrarJugada()
│       └─ calcularPuntosRonda() [matriz_pagos]
├─ calcularEstadisticas()      [estadisticas_torneo]
├─ ordenarRanking()            [estadisticas_torneo]
├─ mostrarDetalleTorneo()      [reporte_resultados]
│   └─ mostrarDetalleEstrategia() x 8
├─ mostrarRanking()            [reporte_resultados]
└─ mostrarGanador()            [reporte_resultados]
```

3.6 Diagrama de flujo del programa principal



4. Análisis de resultados

Número de rondas usado: 250 por partida (8 estrategias, 28 partidas, 7 partidas por estrategia). Los resultados que no involucran a la estrategia Aleatoria son deterministas y se comprobaron a mano (ver Anexo).

4.1 Ranking final

Pos	Estrategia	Puntos		G	P	E	% C	% T
1	Tit for Tat	4553		0	1	6	78.40	21.60
2	Vengativa	4261		2	1	4	57.37	42.63
3	Pavlov (<i>propia</i>)	4184		1	2	4	76.86	23.14
4	Ojo por Dos Ojos	3950		0	3	4	67.94	32.06
5	Sonda (<i>propia</i>)	3828		2	3	2	26.74	73.26
6	Siempre Traicionar	3750		7	0	0	0.00	100.00
7	Aleatoria	3734		3	2	2	50.06	49.94
8	Siempre Cooperar	3408		0	3	4	100.00	0.00

G = partidas ganadas, P = perdidas, E = empatadas. La barra es proporcional al puntaje de la ganadora.

4.2 Puntos de cada estrategia contra cada rival

Cada celda indica los puntos que obtuvo la estrategia de la **fila** contra la de la **columna** (valor de puntosContra).

Fila \ Rival	S. Coop.	S. Trai.	Aleat.	TFT	Veng.	O2O	Pavlov	Sonda	Total
Siempre Cooperar	—	0	402	750	750	750	750	6	3408
Siempre Traicionar	1250	—	726	254	254	258	750	258	3750
Aleatoria	982	131	—	555	130	776	591	569	3734
Tit for Tat	750	249	555	—	750	750	750	749	4553
Vengativa	750	249	755	750	—	750	750	257	4261
Ojo por Dos Ojos	750	248	451	750	750	—	750	251	3950
Pavlov	750	125	556	750	750	750	—	503	4184
Sonda	1246	248	569	749	252	266	498	—	3828

ganó la partida

perdió la partida

empate

Robustez del resultado: la estrategia Aleatoria cambia en cada ejecución, así que el torneo se repitió 30 veces con semillas distintas (250 rondas). Tit for Tat ganó las 30 veces, con puntajes entre 4526 y 4587.

4.3 ¿Cuál fue la estrategia ganadora?

La ganadora fue **Tit for Tat** con **4553 puntos**, 292 puntos por encima de Vengativa (2.º lugar) y 369 por encima de Pavlov (3.º). Lo llamativo es que Tit for Tat **no ganó ninguna partida individual** (0 G, 1 P, 6 E).

4.4 ¿Por qué obtuvo el mayor puntaje?

- **Es amable:** nunca traiciona primero, así que con todas las estrategias que cooperan (Siempre Cooperar, Vengativa, Ojo por Dos Ojos, Pavlov) mantiene la cooperación mutua las 250 rondas y obtiene el máximo posible de 750 puntos en cada partida.
- **Castiga de inmediato:** contra Siempre Traicionar solo pierde la primera ronda (0 puntos) y luego se protege: 249 contra 254. Nunca puede perder una partida por más de 5 puntos.
- **Perdona:** contra la Sonda, que la traiciona en la ronda 1, devuelve la traición una sola vez y en cuanto la Sonda coopera vuelve a cooperar (749 contra 749). Vengativa, en cambio, no perdona esa traición y queda atrapada en traiciones mutuas (257 contra 252), y ahí está la mayor parte de la diferencia entre el 1.º y el 2.º lugar.
- **Es clara:** como copia la jugada anterior, las estrategias reactivas “entienden” rápido que conviene cooperar con ella.

El caso contrario es **Siempre Traicionar**: gana las 7 partidas (nunca saca menos que su rival) pero queda 6.ª, porque contra las estrategias que se defienden solo consigue 1 punto por ronda. **Ganar partidas no es lo mismo que ganar el torneo**: lo que cuenta es el total de puntos, y la cooperación mutua (3 + 3) produce mucho más que la traición mutua (1 + 1).

4.5 ¿Contra qué estrategia obtuvo peores resultados?

Contra **Siempre Traicionar**: 249 puntos, su único resultado perdido (249 – 254). La razón es su regla de la ronda 1: Tit for Tat siempre empieza cooperando y recibe 0 puntos mientras el rival recibe 5. Desde la ronda 2 copia la T del rival y ambos obtienen 1 punto por ronda (249 × 1). Contra un rival que nunca coopera no existe forma de recuperar esa primera ronda. El segundo peor resultado fue contra la **Aleatoria** (555), porque al copiar jugadas al azar alterna entre castigar y ser engañada.

4.6 ¿Las estrategias propias funcionaron como esperábamos?

Pavlov (3.º lugar, 4184 puntos): sí, en gran parte. Cooperar perfectamente con todas las estrategias amables (750 en cuatro partidas) y le gana a la Sonda (503 – 498), porque sale sola de las traiciones mutuas. Su debilidad, prevista en el diseño, apareció contra Siempre Traicionar: después de C-T cambia a T y después de T-T vuelve a C, así que alterna C, T, C, T... y solo obtiene 125 puntos, el peor resultado de todo el torneo entre las estrategias que se defienden. Esa sola partida le cuesta el 2.º lugar.

Sonda (5.º lugar, 3828 puntos): funcionó de forma parcial.

- Contra Siempre Cooperar funcionó exactamente como se esperaba: la detectó como ingenua y la explotó (1246 – 6).
- Contra Tit for Tat logró volver a la cooperación (749 – 749).
- Contra **Ojo por Dos Ojos** hubo un resultado inesperado: esta perdona una traición aislada, así que no se “defendió” en las rondas 2 y 3 y la Sonda pasó a modo explotar. Pero Ojo por Dos Ojos sí responde cuando recibe dos traiciones seguidas, y desde la ronda 6 la partida quedó en traición mutua. La Sonda ganó la partida (266 – 251), pero obtuvo 484 puntos menos que los 750 que habría conseguido

cooperando. El error de diseño es que el modo explotar es permanente y no revisa si el rival empezó a defenderse después.

- La traición de prueba de la ronda 1 fue muy costosa contra **Vengativa**: esta nunca perdona, así que la partida quedó en traición mutua desde la ronda 3 (252 – 257). Contra Pavlov entró en un ciclo C-T / T-C / T-T que da 6 puntos cada 3 rondas (498).

La conclusión es que “probar” al rival tiene un costo: la ganancia contra Siempre Cooperar (unos +500 puntos respecto a cooperar) no compensa lo que se pierde contra las estrategias que castigan (unos –500 contra Vengativa, –480 contra Ojo por Dos Ojos y –250 contra Pavlov). Una mejora posible sería que, en modo explotar, la Sonda volviera al modo Tit for Tat si el rival traiciona dos veces seguidas.

5. Conclusiones

1. En el Dilema del Prisionero Iterado la cooperación es más rentable que la traición: las cuatro primeras estrategias del ranking son “amables” (nunca traicionan primero), y la que más traiciona, Siempre Traicionar, queda en el 6.º lugar a pesar de ganar todas sus partidas.
2. Tit for Tat ganó porque combina cuatro cualidades: es amable, castiga de inmediato, perdona rápido y es fácil de entender. Esto coincide con los resultados de los torneos de Robert Axelrod.
3. El perdón es importante: la diferencia principal entre Tit for Tat (1.º) y Vengativa (2.º) está en la partida contra la Sonda, donde una sola traición llevó a Vengativa a castigar para siempre.
4. Las estrategias que intentan explotar (Sonda) ganan mucho contra rivales ingenuos, pero pierden más contra rivales que castigan. En un torneo con estrategias variadas no conviene traicionar primero.
5. En cuanto a la programación, la separación en módulos, la `struct Estrategia` y el puntero a función `FuncionDecision` permitieron que `ejecutarPartida` funcione con cualquier estrategia sin `if/switch`, y que agregar una estrategia nueva solo requiera escribir su función y registrarla en `inicializarEstrategias`.

6. Bibliografía y fuentes

- Axelrod, R. (1984). *The Evolution of Cooperation*. Basic Books.
- Nowak, M., & Sigmund, K. (1993). A strategy of win-stay, lose-shift that outperforms tit-for-tat in the Prisoner's Dilemma game. *Nature*, 364, 56–58.
- Enunciado del Proyecto 1 y material del curso Programación II — UTEC.
- Herramientas de IA: se usó un asistente de IA (Claude, de Anthropic) como apoyo para organizar el informe y verificar los resultados. Todos los resultados fueron comprobados ejecutando el programa y con los cálculos a mano del Anexo.

Anexo: compilación y pruebas

```
g++ -std=c++11 -Wall -o torneo_dilema main_torneo.cpp lectura_rondas.cpp estrategias_juego.cpp \
    matriz_pagos.cpp partida_iterada.cpp torneo_round_robin.cpp estadisticas_torneo.cpp
reporte_resultados.cpp
./torneo_dilema          (o bien:  make run)
```

Validación de la entrada

Entrada	Resultado obtenido
abc	"Entrada invalida. Debe ingresar un numero entero." y vuelve a preguntar.
100	"El numero de rondas debe ser mayor a 200." y vuelve a preguntar.
200	Mismo mensaje (200 no es mayor a 200).
250	Se acepta y se ejecuta el torneo.
5000 , 100000000	Se aceptan (no hay máximo) y el torneo termina correctamente. Con 100 000 000 rondas tarda unos 26 s y Tit for Tat suma 1 824 990 479 puntos; el puntaje posible llega a 3 500 000 000, por eso se usa long long .

Partidas deterministas comprobadas a mano (250 rondas)

Partida	Cálculo	Esperado	Programa
Siempre Traicionar vs Siempre Cooperar	$250 \times 5 / 250 \times 0$	1250 / 0	1250 / 0
Tit for Tat vs Siempre Traicionar	$0 + 249 \times 1 / 5 + 249 \times 1$	249 / 254	249 / 254
Ojo por Dos Ojos vs Siempre Traicionar	$0 + 0 + 248 \times 1 / 5 + 5 + 248$	248 / 258	248 / 258
Pavlov vs Siempre Traicionar	$125 \times 0 + 125 \times 1 / 125 \times 5 + 125 \times 1$	125 / 750	125 / 750
Sonda vs Siempre Cooperar	$5 + 3 + 3 + 247 \times 5$	1246 / 6	1246 / 6
Sonda vs Tit for Tat	$5 + 0 + 248 \times 3$	749 / 749	749 / 749
Sonda vs Vengativa	$5 + 0 + 0 + 247 \times 1$	252 / 257	252 / 257

Comprobaciones generales

- 28 partidas en total; en cada fila del ranking $G + P + E = 7$.
- $\text{vecesC} + \text{vecesT} = 7 \times 250 = 1750$ para cada estrategia, y $\% C + \% T = 100$.
- Siempre Cooperar tiene 100 % de C y Siempre Traicionar 100 % de T; la Aleatoria queda cerca del 50 %.